

1 Carbon-Aware Scheduling: MILP Formulation

1.1 Problem Definition

We formulate the carbon-aware scheduling problem as a Mixed Integer Linear Programming (MILP) problem. The objective is to find an optimal assignment of pods (computational tasks) to nodes (computing resources) and timeslots while minimizing the total carbon emissions.

1.2 Sets and Indices

$i \in \mathcal{P}$	Set of pods	(1)
$j \in \mathcal{F}$	Set of flavours (nodes)	(2)
$t \in \mathcal{T}$	Set of timeslots	(3)
		(4)

1.3 Parameters

$d_i \in \mathbb{Z}^+$	Duration of pod i (in timeslots)	(5)
$c_i \in \mathbb{R}^+$	CPU request of pod i	(6)
$r_i \in \mathbb{R}^+$	RAM request of pod i	(7)
$s_i \in \mathbb{Z}^+$	Earliest timeslot for pod i	(8)
$C_{jt} \in \mathbb{R}^+$	Available CPU of node j in timeslot t	(9)
$R_{jt} \in \mathbb{R}^+$	Available RAM of node j in timeslot t	(10)
$e_{ijt} \in \mathbb{R}^+$	Carbon emissions for running pod i on node j starting at timeslot t	(11)
$T_{max} \in \mathbb{Z}^+$	Maximum available timeslot index	(12)
		(13)

The carbon emissions e_{ijt} are calculated considering:

- Operational carbon: Based on the pod's resource consumption, node's power profile, and regional carbon intensity forecasts across the pod's execution duration

- Embodied carbon: Accounting for the amortized manufacturing emissions of the hardware over its lifetime

1.4 Decision Variables

$$x_{ijt} = \begin{cases} 1 & \text{if pod } i \text{ is placed on node } j \text{ starting at timeslot } t \\ 0 & \text{otherwise} \end{cases} \quad (14)$$

1.5 Feasible Placements

Let $\mathcal{V}_i \subset \mathcal{F} \times \mathcal{T}$ be the set of valid node-timeslot pairs for pod i :

$$\mathcal{V}_i = \{(j, t) \in \mathcal{F} \times \mathcal{T} \mid t \geq s_i \text{ and} \quad (15)$$

$$t + d_i \leq T_{max} \text{ and} \quad (16)$$

$$C_{j(t+k)} \geq c_i \text{ for all } k \in \{0, 1, \dots, d_i - 1\} \text{ and} \quad (17)$$

$$R_{j(t+k)} \geq r_i \text{ for all } k \in \{0, 1, \dots, d_i - 1\}\} \quad (18)$$

1.6 Mathematical Model

1.6.1 Objective Function

Minimize total carbon emissions:

$$\text{Minimize } Z = \sum_{i \in \mathcal{P}} \sum_{(j,t) \in \mathcal{V}_i} e_{ijt} \cdot x_{ijt} \quad (19)$$

1.6.2 Constraints

Assignment constraint: Each pod must be placed exactly once:

$$\sum_{(j,t) \in \mathcal{V}_i} x_{ijt} = 1 \quad \forall i \in \mathcal{P} \quad (20)$$

CPU capacity constraints: The total CPU usage in each node and timeslot must not exceed capacity:

$$\sum_{i \in \mathcal{P}} \sum_{t' \in \mathcal{T}_{it}} c_i \cdot x_{ijt'} \leq C_{jt} \quad \forall j \in \mathcal{F}, \forall t \in \mathcal{T} \quad (21)$$

where $\mathcal{T}_{it} = \{t' \in \mathcal{T} \mid t' \leq t < t' + d_i, (j, t') \in \mathcal{V}_i\}$ represents the set of starting timeslots t' such that pod i would be running at timeslot t if started at t' .

RAM capacity constraints: The total RAM usage in each node and timeslot must not exceed capacity:

$$\sum_{i \in \mathcal{P}} \sum_{t' \in \mathcal{T}_{it}} r_i \cdot x_{ijt'} \leq R_{jt} \quad \forall j \in \mathcal{F}, \forall t \in \mathcal{T} \quad (22)$$

Binary constraint: Decision variables are binary:

$$x_{ijt} \in \{0, 1\} \quad \forall i \in \mathcal{P}, \forall (j, t) \in \mathcal{V}_i \quad (23)$$

1.7 Emissions Calculation

The carbon emissions parameter e_{ijt} for running pod i on node j starting at timeslot t is calculated as:

$$e_{ijt} = \sum_{k=0}^{d_i-1} (op_{ijt+k} + emb_{ij}) \quad (24)$$

where:

- op_{ijt+k} is the operational carbon emissions in timeslot $t+k$:

$$op_{ijt+k} = p_{ij} \cdot CI_{j,t+k} \quad (25)$$

with p_{ij} being the power consumption of pod i on node j and $CI_{j,t+k}$ being the carbon intensity forecast for the region of node j in timeslot $t+k$.

- emb_{ij} is the embodied carbon emissions attributed to pod i on node j for one timeslot:

$$emb_{ij} = \frac{EC_j}{L_j \cdot \text{utilization factor}} \quad (26)$$

with EC_j being the total embodied carbon of node j , L_j being the node's expected lifetime in hours, and the utilization factor representing the pod's resource usage as a fraction of the node's total capacity.

1.8 Implementation Details

In practice, the problem is solved using the PuLP library with the following approach:

1. Determine the earliest timeslot (s_i) for each pod based on its source file
2. Precompute all valid placements for each pod, respecting the earliest timeslot constraint
3. Create binary decision variables for each valid placement
4. Define the objective function to minimize total carbon emissions
5. Add constraints ensuring each pod is placed exactly once
6. Add resource capacity constraints for CPU and RAM
7. Solve the MILP problem using the CBC solver with a time limit
8. Extract the solution from the decision variables

1.8.1 Earliest Timeslot Constraint

The earliest timeslot constraint ($t \geq s_i$) ensures that pods are only scheduled at or after their designated earliest timeslot. This constraint is derived from the pod's source file (timeslot_X.yaml), where X indicates the earliest timeslot at which the pod can be scheduled. This approach prevents pods from being scheduled before they are available in the system, reflecting real-world operational requirements.